

Design Document

Goal/Overview: Implementing a binary tree that is going to capture information about the main tributaries of the Columbia River.

Modules:

1. Add body of water: (Add a body of water to the JSON file)
2. Navigate Watershed: (Go through the watershed using a choice from the user through where to navigate)
3. Display entire watershed: (Display the watershed and basically the tree to map out the watershed)
4. Save to file: (Save changes like additions to the json)
5. Load from file: (Load the information from the json)
6. Exit: (Exit program)

Enter Choice:

Utilities

1. Binary Search Tree: Used to navigate through data and go through watersheds to pull information in a quick and efficient way.
2. JSON: Used to store all data and to manipulate data when needed in an efficient way.

Designs

For the Dams, we will be making those as structs and attaching them to their associated water body node.

We have constructed the tree to be in a orderly way in which the user can choose wherever to go left, right, or to the parent of a selected node. Thus the input will navigate through the given JSON file to be able to give information about each tributary necessary,

A search tree was used because for the sake of the program, it was the most useful and reliable form of organizing the data in a particular way that makes it easy for the user to navigate through the tributaries and have a map of sorts to help understand positioning.

Nodes were used for the tributaries themselves to separate and locate data in a fast manner, where dams were stored in a vector for a compact and ordered manner.

How to run

First, it is important to download the JSON extension for the code to actually run on a given device. Under Make Help, we see a URL given to directly download the necessary content to run the following program

Ubuntu/Debian:

```
sudo apt-get install nlohmann-json3-dev
```

macOS:

```
brew install nlohmann-json
```

Manual installation:

```
wget https://github.com/nlohmann/json/releases/download/v3.11.3/json.hpp
sudo mkdir -p /usr/local/include/nlohmann
sudo mv json.hpp /usr/local/include/nlohmann/
```

To run the application, simply run Make. This will provide the user with a menu where they can navigate, search, or manipulate the tree of watershed as they please.

To run the unit tests, simply run Make test and the program will go through each of the unit tests in the Tests/ directory.

Use of AI

For this project, AI was used for the purpose of making the program more efficient and cost effective. It was also utilized for cleanliness and organization of code. AI was also used for debugging or fixing code that as a group we had issues fixing ourselves.